



RecLib: a blazingly fast, principled recommendation systems Rust library

Miguel Martínez

Pontificia Universidad Católica de Chile

Motivación y Objetivos

Por sus características, **Rust** [5] es un lenguaje idóneo para el diseño y construcción de software que exija alto rendimiento y correctitud:

- **Memory safety:** sin *garbage collector*, garantizada por el *borrow checker*.
- **Fearless concurrency:** las *data races* ocurren en tiempo de compilación, no de ejecución.
- **Zero-cost abstractions:** rendimiento igual a C/C++ sin renunciar a código expresivo.
- Ecosistema numérico emergente: **ndarray**, **sprs**, **rayon** y **arrow** como *building blocks*.

Exponer **bindings a Python** es *absolutamente necesario* para mantener la velocidad de prototipado, especialmente en flujos de trabajo inherentemente iterativos como los de Sistemas Recomendadores y Machine Learning en general. El ecosistema de Rust ofrece herramientas para definir este tipo de FFI sin fricción mediante **PyO3** [7].

Por qué recomendación secuencial

Se decidió iniciar el desarrollo en modelos del paradigma de **recomendación secuencial** (basada en sesiones) por su **relevancia y vigencia**: es un paso más allá de tratar las interacciones como un conjunto estático (filtrado colaborativo clásico), modela el *orden* y la intención a corto plazo de los usuarios para predecir el **siguiente ítem** a partir del contexto reciente. Es un enfoque importante en dominios reales como el comercio electrónico y *streaming*, donde el usuario suele ser anónimo. La familia SKNN sigue siendo una baseline muy competitiva.

Oportunidad

No existen **librerías modernas de sistemas recomendadores escritas en Rust**: están los *building blocks*, pero nadie los ha integrado en una biblioteca cohesionada. **RecLib** provee (*i.*) un core en Rust, seguro (`#![forbid(unsafe_code)]`), rápido y reproducible; (*ii.*) bindings a Python vía PyO3 sin sacrificar velocidad; y (*iii.*) correctitud y performance validados.

Modelos y Métricas

Modelos soportados

- **Popularity**; baseline por frecuencia.
- **Random**; baseline determinista (*seeded*) por azar.
- **SKNN**; kNN basado en sesiones (*cosine similarity*).
- **S-SKNN**; variante secuencial por posición.
- **V-SKNN**; variante ponderada por *recency* de la posición.

Métricas de ranking

HR@k, Precision@k, Recall@k, F1@k, MRR, nDCG@k, MAP y ARHR@k. Las métricas se encuentran desacopladas de los modelos; la evaluación *leave-one-out* se expone a través del método `evaluate_leave_one_out`.

Un ejemplo en pocas líneas

```
from reclib import VSknn, SknnConfig

# Entrenar V-SKNN sobre sesiones (listas de item ids)
model = VSknn.fit(sessions, SknnConfig(k_neighbors=100))

# Recomendar los 20 items siguientes para un prefijo
recs = model.recommend(prefix, k=20)

# Evaluacion leave-one-out sobre el conjunto de test
report = model.evaluate_leave_one_out(test, k=20)
print(report.hit_rate, report.mrr, report.ndcg)
```

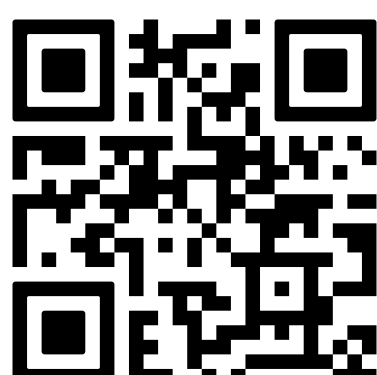
Código y Documentación



Repositorio
gitlab.com/reclib



Documentación
reclib.gitlab.io



Cookbook
marimo notebook

Arquitectura de RecLib

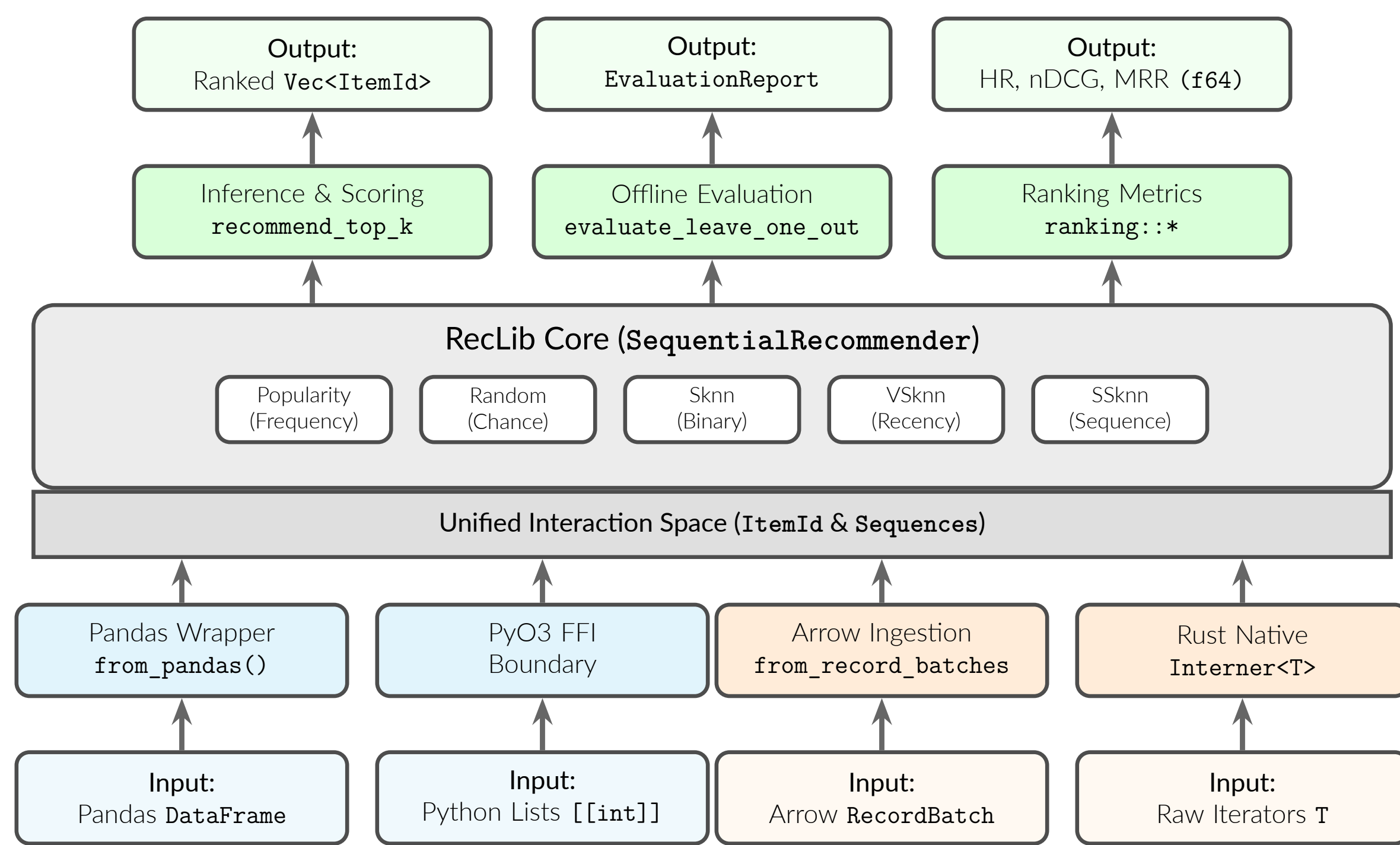


Figure 1. Arquitectura de RecLib: múltiples *frontends* convergen sobre un mismo core.

Cuatro alternativas de ingestión de datos: (1.) **DataFrame** de Pandas, (2.) listas de Python (vía la FFI de **PyO3**), (3.) **RecordBatch** de Arrow e (4.) iterators nativos de Rust; convergen en un **espacio de interacción unificado** (**ItemId** y **Sequences**). Sobre él, **SequentialRecommender**, el *trait* core de la librería, define la interfaz común de todos los modelos, y las operaciones de **inferencia**, **evaluación** y **métricas** producen las salidas. La misma lógica sirve idénticamente a Python y a Rust: no existen reimplementaciones por lenguaje.

Pipeline de Artefactos y Distribución

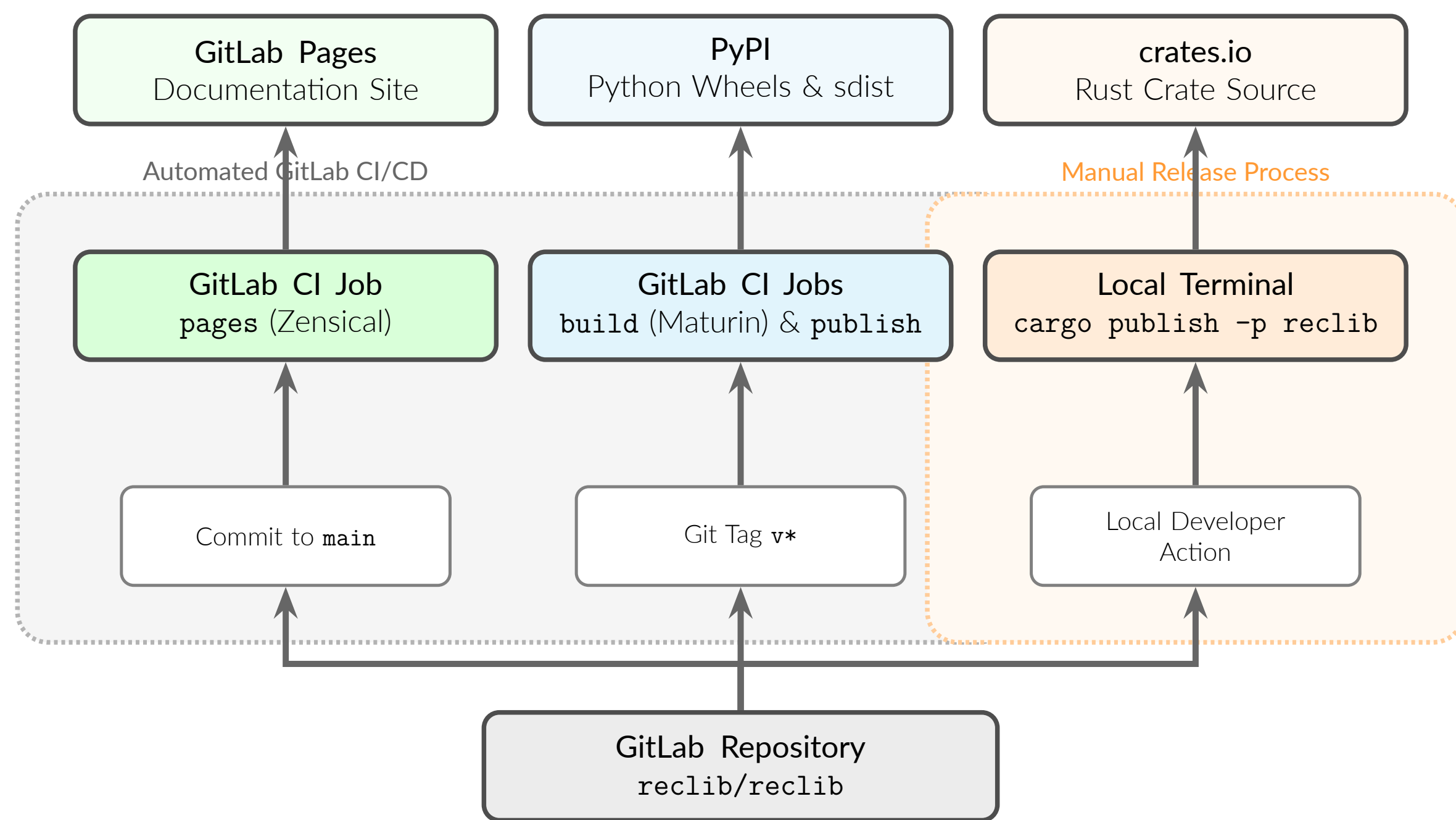


Figure 2. Diagrama de distribución de RecLib.

El código fuente se encuentra hospedado en un repositorio de GitLab. De él, se derivan tres artefactos versionados: (*i.*) la **documentación** (Zensical), publicada en GitLab Pages por CI al hacer *commit* a la rama **main**; (*ii.*) los **wheels** de Python, compilados con **Maturin** y publicados en **PyPI** al taggear una versión; y (*iii.*) el **crate** de Rust, publicado manualmente en **crates.io** vía Cargo.

Benchmark

Se comparó RecLib contra dos referencias que implementan la misma familia de modelos: **session-rec** [4] (Ludewig & Jannach) y **Serenade** [3] (bol.com, VMIS-kNN en Rust). **session-rec** es el **punto de comparación canónico**: es el *framework* de referencia que implementa exactamente la familia SKNN, V-SKNN y S-SKNN [2]; actúa como **oráculo de correctitud**. Por su parte, Serenade aporta un marco comparativo para el rendimiento máximo esperado de una implementación de Rust.

Mediciones

- **Correctitud**: equivalencia algorítmica (misma configuración entre librerías); paridad exacta de *ranking* (muestreo desactivado); paridad de métricas dentro de tolerancia.
- **Performance**: velocidad de entrenamiento (*fit*) e inferencia (*throughput* y latencia p50/p95/p99). RecLib se mide **dos veces**: nativo en Rust (techo) y vía PyO3 (comparación justa con Python).

Datos idénticos (MovieLens 100K [1]), misma configuración, threads fijos y mediana de repeticiones. Se utilizó *containerization* vía **Docker** para asegurar reproducibilidad.

Resultados

Correctitud; métricas de ranking

Modelo	Biblioteca	HR@20	MRR@20	nDCG@20
SKNN	RecLib	0.0947	0.0149	0.0316
SKNN	session-rec	0.0947	0.0149	0.0316
V-SKNN	RecLib	0.1187	0.0215	0.0419
V-SKNN	session-rec	0.0747	0.0134	0.0265
S-SKNN	RecLib	0.1480	0.0330	0.0578
S-SKNN	session-rec	0.1280	0.0211	0.0437

Table 1. Métricas de *ranking* en MovieLens 100K ($n = 750$, corte @20).

SKNN: **98.5 %** de *rankings* idénticos entre **RecLib** y **session-rec**.

Performance: V-SKNN

Biblioteca	Fit (ms)	Cons./s	p95 (ms)	Speedup
RecLib (Rust)	1.2	6706	0.28	7840×
RecLib (PyO3)	3.3	1151	1.2	1345×
session-rec	57.3	0.86	4079	1.0×
Serenade [†]	<i>n.d.</i>	<i>n.d.</i>	0.43	—

Table 2. Rendimiento de V-SKNN en MovieLens 100K ($n = 200$, mediana de 5 repeticiones, un thread, muestreo desactivado). *Speedup* de *throughput* relativo a **session-rec**. [†]Serenade: la latencia por predicción de su **evaluator** impide que ejercite su ventaja, diseñada para una mayor cantidad de datos.

Conclusiones y Trabajo Futuro

Mientras **SKNN** alcanza paridad casi exacta con **session-rec** (98.5 % de *rankings* idénticos), **V-SKNN** y **S-SKNN** presentan divergencia. Se atribuye a un **desajuste de configuración** en: la **ponderación por recency** dentro de la sesión y/o el **decaimiento del score** de los **ítems candidatos**. Reconciliar estas funciones y discernir si se trata de un mal emparejamiento de hiperparámetros (paridad exacta alcanzable) o de una diferencia de definición (equivalencia aproximada, con Δ pequeño) es el paso inmediato para cerrar la comparación de correctitud de la familia completa.

Este trabajo constituye una **base sólida** para una biblioteca que demuestre las capacidades de un tooling moderno: un core seguro y testeado, la rapidez de ejecución de Rust combinada con la rapidez de iteración de Python, correctitud validada contra el estándar del área sin pérdidas de rendimiento. Los siguientes pasos son la incorporación de **FPMC** [6] y modelos neuronales secuenciales como **GRU4Rec** y **SASRec**. También se espera incluir datasets out-of-the-box.

Referencias

- [1] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015.
- [2] Dietmar Jannach and Malte Ludewig. When recurrent neural networks meet the neighborhood for session-based recommendation. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (RecSys '17)*, pages 306–310. ACM, 2017.
- [3] Barrie Kersbergen, Olivier Sprangers, and Sebastian Schelter. Serenade – low-latency session-based recommendation in e-commerce at scale. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*, pages 150–159. ACM, 2022.
- [4] Malte Ludewig and Dietmar Jannach. Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction*, 28(4–5):331–390, 2018.
- [5] Nicholas D. Matsakis and Felix S. Klock II. The Rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT '14)*, pages 103–104. ACM, 2014.
- [6] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. Factorizing personalized Markov chains for next-basket recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW '10)*, pages 811–820. ACM, 2010.
- [7] The PyO3 Project. PyO3: Rust bindings for Python. <https://github.com/PyO3/pyo3>, 2024.